

Scientific Software Design

Presented by



Members of



*Consortium for the Advancement
of Scientific Software*
<https://cass.community>

With prior support from



Anshu Dubey (she/her)
Argonne National Laboratory

Better Software for Science with High-Performance Computing tutorial
@ SupercomputingAsia 2026 (SCA 2026)/The International Conference
on High Performance Computing in Asia-Pacific Region 2026 (HPCAsia
2026) conference

Contributors: Anshu Dubey (ANL), David E. Bernholdt (ORNL)



See slide 2 for
license details

License, Citation and Acknowledgements

License and Citation



- This work is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/) (CC BY 4.0).
- **The requested citation the overall tutorial is:** Anshu Dubey and Akash Dhruv, Better Software for Science with High-Performance Computing tutorial, in SupercomputingAsia 2026 (SCA 2026)/The International Conference on High Performance Computing in Asia-Pacific Region 2026 (HPCAsia 2026), Osaka, Japan, 2026. DOI: [10.6084/m9.figshare.31130062](https://doi.org/10.6084/m9.figshare.31130062).
- Individual modules may be cited as *Speaker, Module Title, in Tutorial Title, ...*

Acknowledgements

- This work was supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Next-Generation Scientific Software Technologies (NGSST) and Scientific Discovery through Advanced Computing (SciDAC) programs.
- This work was supported by the U.S. Department of Energy Office of Science, Office of Advanced Scientific Computing Research (ASCR), and by the Exascale Computing Project (17-SC-20-SC), a collaborative effort of the U.S. Department of Energy Office of Science and the National Nuclear Security Administration.
- This work was performed in part at the Argonne National Laboratory, which is managed by UChicago Argonne, LLC for the U.S. Department of Energy under Contract No. DE-AC02-06CH11357.
- This work was performed in part at the Lawrence Livermore National Laboratory, which is managed by Lawrence Livermore National Security, LLC for the U.S. Department of Energy under Contract No. DE-AC52-07NA27344.
- This work was performed in part at the Los Alamos National Laboratory, which is managed by Triad National Security, LLC for the U.S. Department of Energy under Contract No.89233218CNA000001
- This work was performed in part at the Oak Ridge National Laboratory, which is managed by UT-Battelle, LLC for the U.S. Department of Energy under Contract No. DE-AC05-00OR22725.
- This work was performed in part at Sandia National Laboratories. Sandia National Laboratories is a multi-mission laboratory managed and operated by National Technology and Engineering Solutions of Sandia, LLC., a wholly owned subsidiary of Honeywell International, Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525.

Introduction

- Investing some thought in design of software makes it possible to maintain, reuse and extend it
- Even if some research software begins its life as a one-off use case, it often gets reused
 - Without proper design it is likely to accrete features haphazardly and become a monstrosity
 - Acquires a lot of technical debt in the process

“Technical debt – or code debt – is the consequence of software development decisions that result in prioritizing speed or release over the [most] well-designed code,” Duensing says. “It is often the result of using quick fixes and patches rather than full-scale solutions.”

definition from <https://enterpriseproject.com/article/2020/6/technical-debt-explained-plain-english>
 - Many projects have had this happen
 - Most end up with a hard reset and start over again
- In this module we will cover general design principles and those that are tailored for scientific software
- We will also work through two use cases

Designing Software – High Level Phases

Requirements gathering

- ☐ Features and capabilities
- ☐ Constraints
- ☐ Limitations
- ☐ Target users
- ☐ Other

Decomposition

- ☐ Understand design space
- ☐ Decompose into high level components
- ☐ Bin components into types

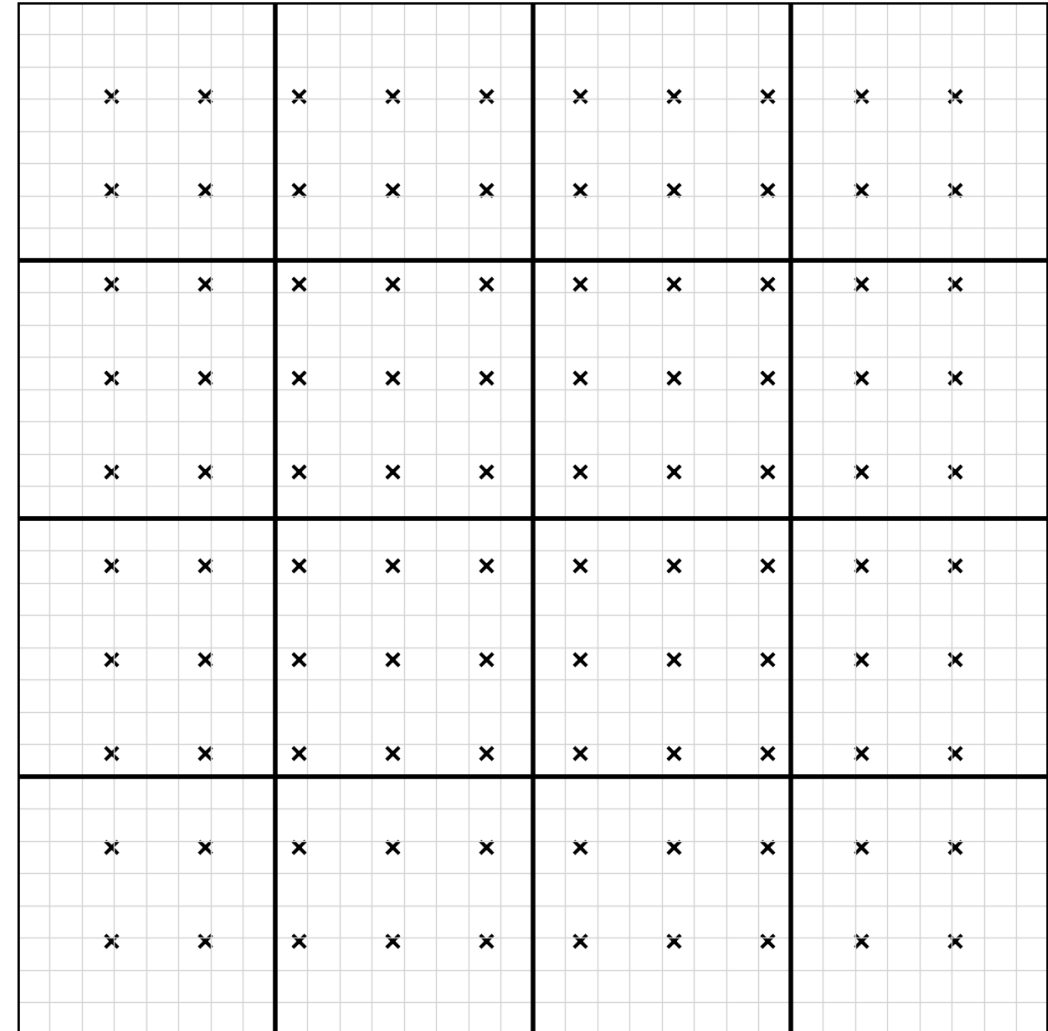
Connectivity

- ☐ Understand component hierarchy
- ☐ Figure out connectivity among components
- ☐ Articulate dependencies

Simple Use Case

- Many applications combine mesh and particles
 - Particles have positions associated with them
 - They interchange physical quantities with the mesh
 - They change positions based on interactions
- We will build a uniformly discretized 2D cartesian mesh and initialize with a lattice of particles
 - We will divide the mesh into blocks
 - We will associate each particle with a single block
 - We will move particles to different locations which may change the block they are associated with
 - We will need to handle particles that leave the domain
- To make sure that we are doing it correctly we need
 - Ways of verifying that particle is associated with the right block

(64.0,64.0)

 $(0,0)$

Requirements gathering

- Components we need
 - Mesh generator – with mesh divided into blocks
 - Particles generator – in a 2D lattice
 - For each particle ability to locate the block on which the particle resides
 - Making the particles change position
- To verify that we are doing it correctly we need to make sure that
 - The mesh is generated correctly
 - The particles are initialized in a lattice and associated with blocks correctly
 - After moving the particles change their associations correctly

Decomposition

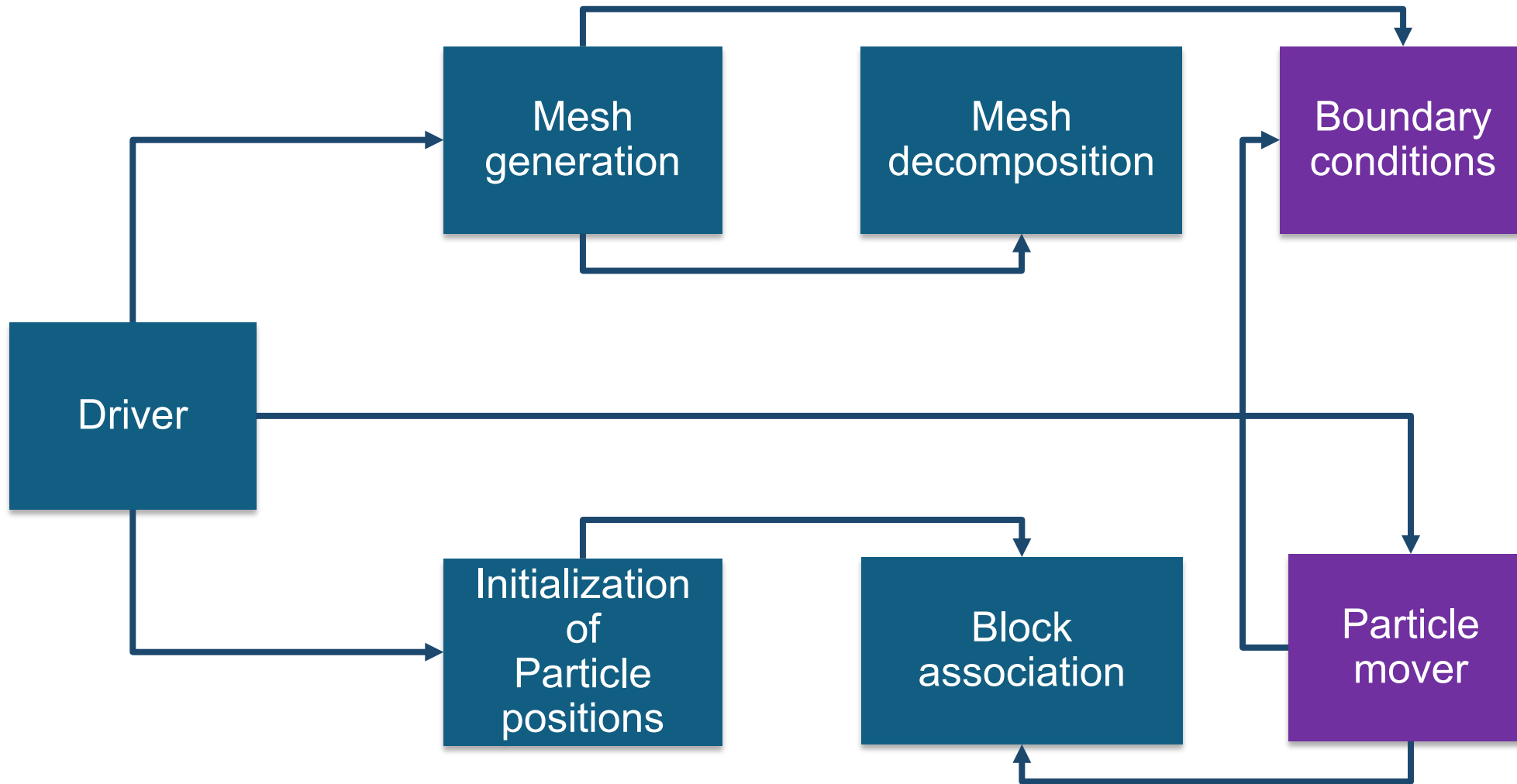
This is a small design space

- ❑ Several requirements can directly map to components
 - ❑ Driver
 - ❑ Mesh initialization
 - ❑ Data structure
 - ❑ Division into blocks
 - ❑ Particle initialization
 - ❑ Position
 - ❑ Association with a mesh block
 - ❑ Particle mover
 - ❑ Tests

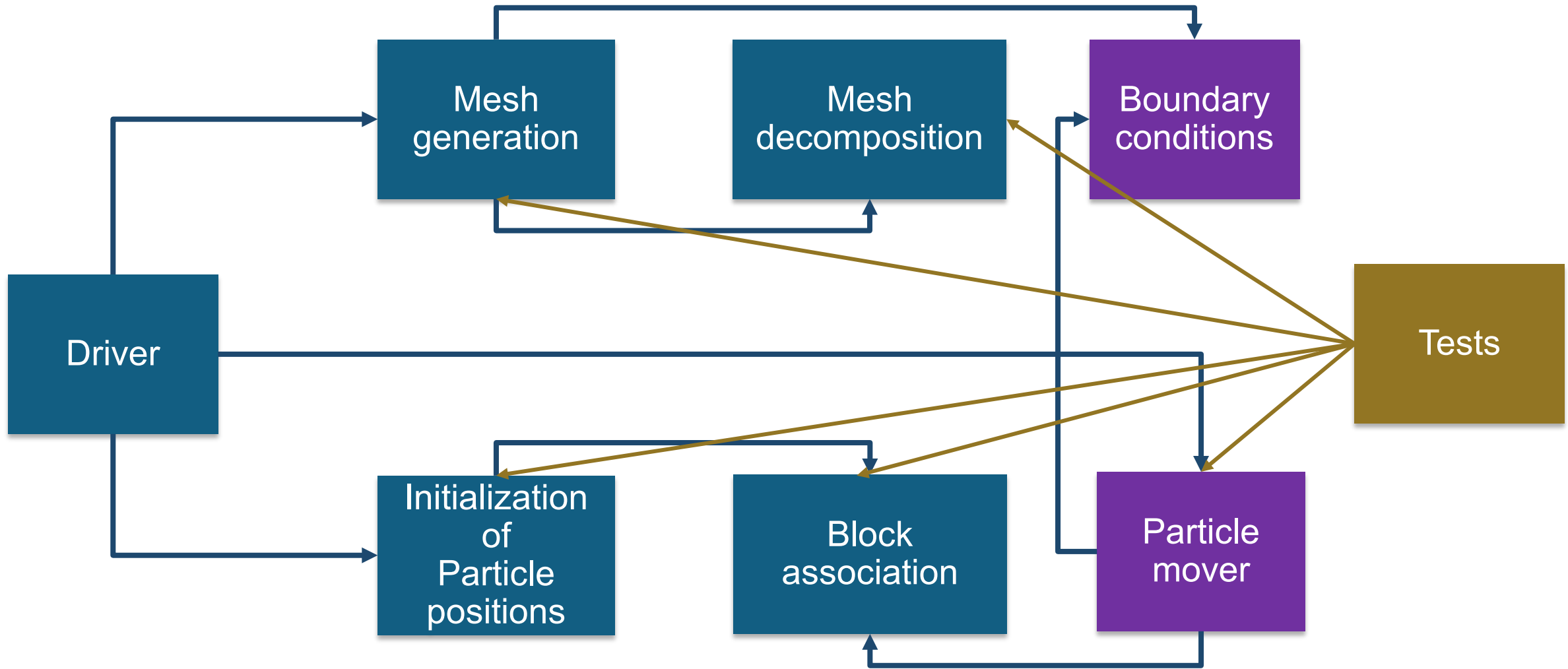
Binning components

- ❑ Infrastructure Components
 - ❑ Driver
 - ❑ Initialization
 - ❑ Mesh with division into blocks
 - ❑ Particles position
 - ❑ Association with a mesh block
- ❑ Components that are specific to one instance
 - ❑ Boundary conditions
 - ❑ Particle mover
- ❑ Components for verification

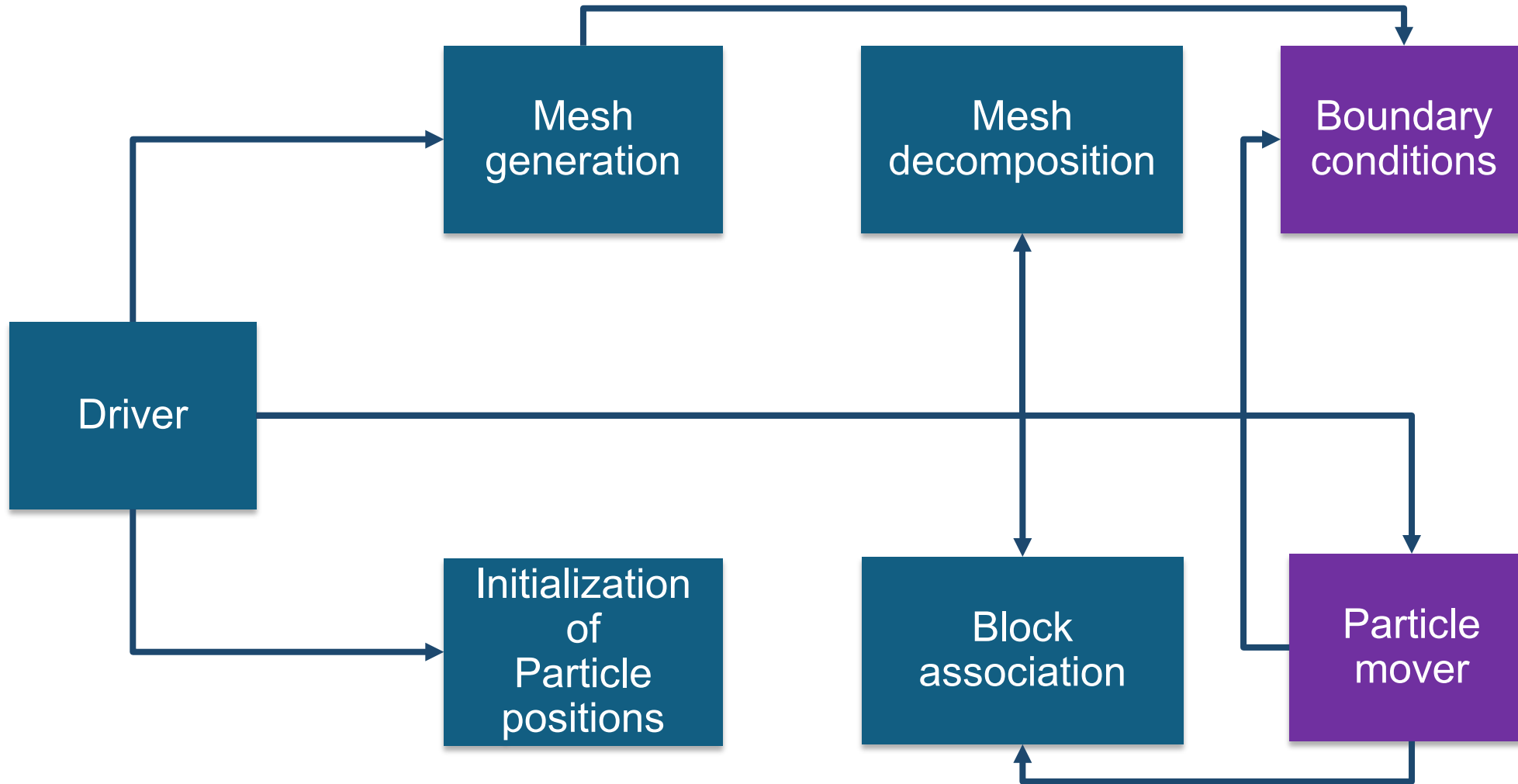
Connectivity



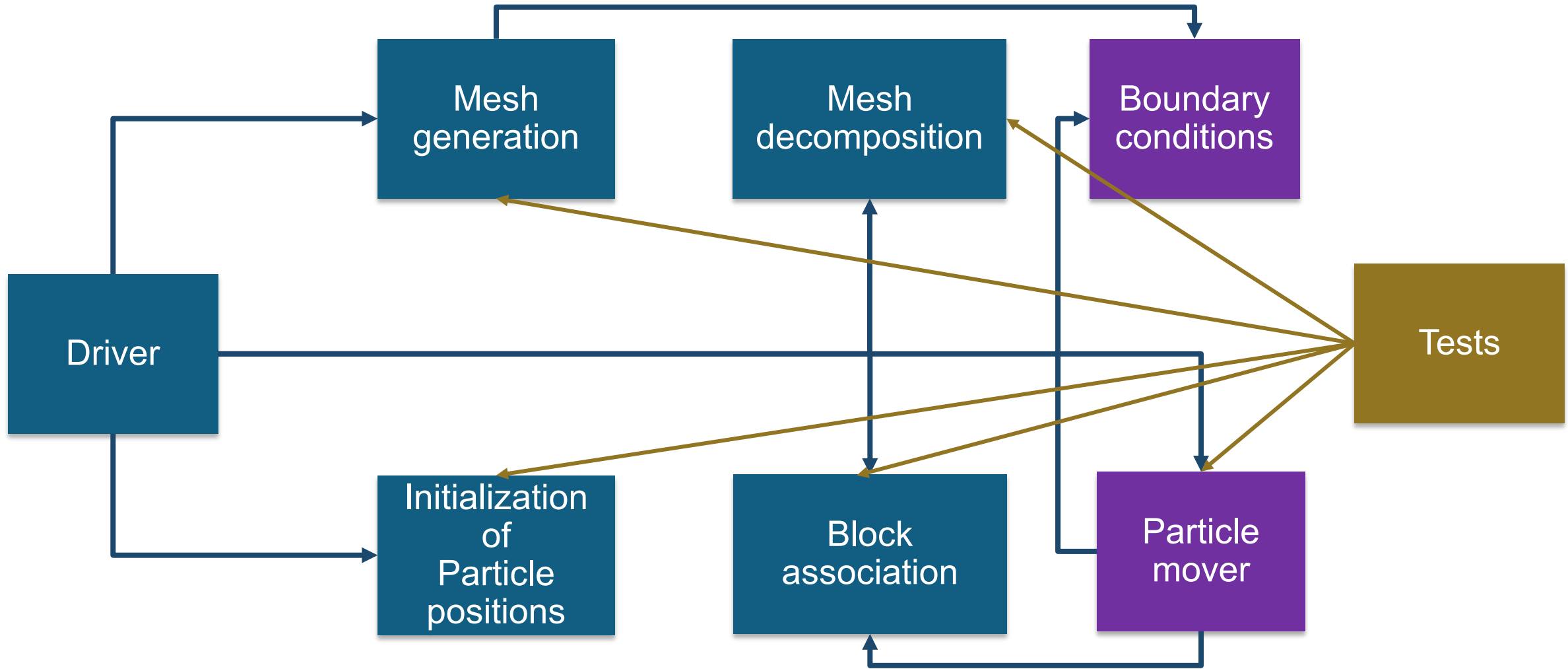
Connectivity



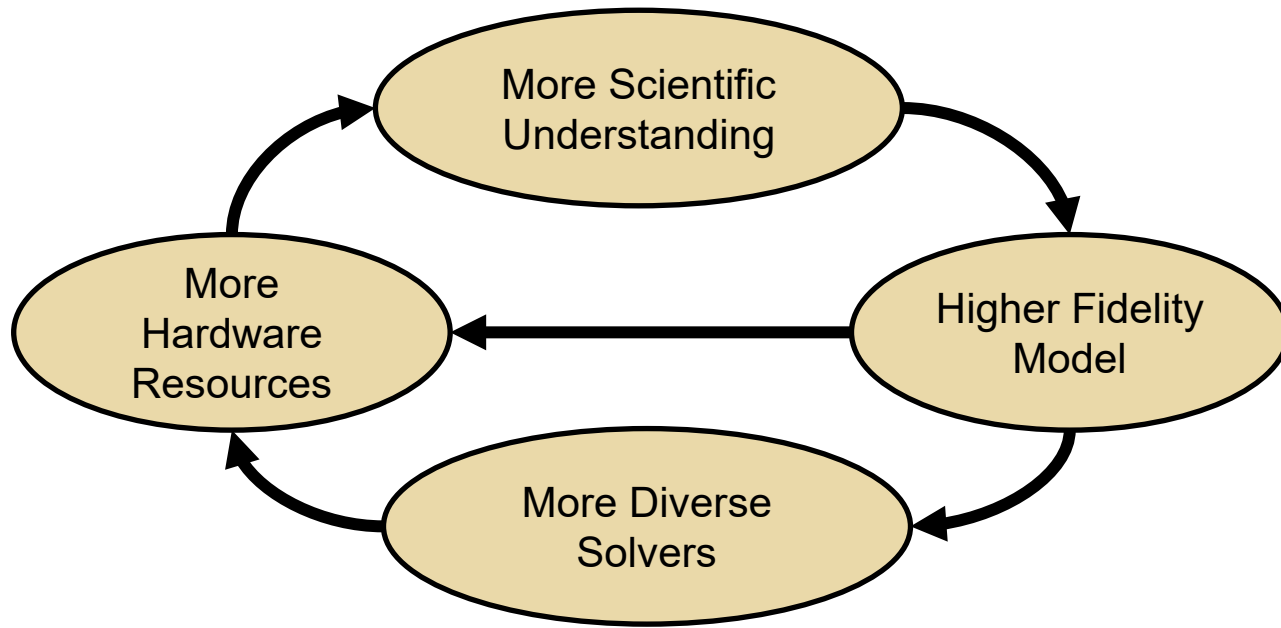
Connectivity



Connectivity



Research Software Challenges



- Many parts of the model and software system can be under research
- Requirements change throughout the lifecycle as knowledge grows
- Verification complicated by floating point representation
- Real world is messy

Additional Considerations for Research Software

Considerations

- ❑ Multidisciplinary
 - ❑ Many facets of knowledge
 - ❑ To know everything is not feasible
- ❑ Two types of code components
 - ❑ Infrastructure (mesh/IO/runtime ...)
 - ❑ Science models (numerical methods)
- ❑ Codes grow
 - ❑ New ideas => new features
 - ❑ Code reuse by others

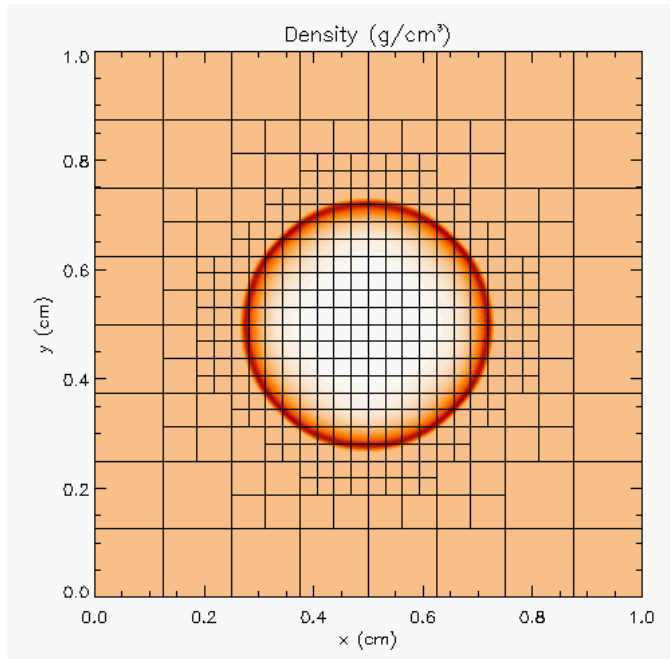
Design Implications

- ❑ Separation of Concerns
 - ❑ Shield developers from unnecessary complexities
- ❑ Work with different lifecycles
 - ❑ Long-lasting vs quick changing
 - ❑ Logically vs mathematically complex
- ❑ Extensibility built in
 - ❑ Ease of adding new capabilities
 - ❑ Customizing existing capabilities

More Complex Application Design – Sedov Blast Wave

Description

High pressure at the center cause a shock to moves out in a circle. High resolution is needed only at and near the shock



Requirements

- Adaptive mesh refinement
 - Easiest with finite volume methods
- Driver
- I/O
- Initial condition
- Boundary condition
- Shock Hydrodynamics
- Ideal gas equation of state
- Method of verification

Deeper Dive into Requirements

- Adaptive mesh refinement → divide domain into blocks
 - Blocks need halos to be filled with values from neighbors or boundary conditions
 - At fine-coarse boundaries there is interpolation and restriction
 - Blocks are dynamic, go in and out of existence
 - Conservation needs reconciliation at fine-coarse boundaries
- Shock hydrodynamics
 - Solver for Euler's equations at discontinuities
 - EOS provides closure
 - Riemann solver
 - Halo cells are fine-coarse boundaries need EOS after interpolation
- Method of verification
 - An indirect way of checking – shock distance traveled can be computed analytically

Components

Binned Components

☐ Unchanging or slow changing infrastructure

- ☐ Mesh
- ☐ I/O
- ☐ Driver
- ☐ Comparison utility -- testing

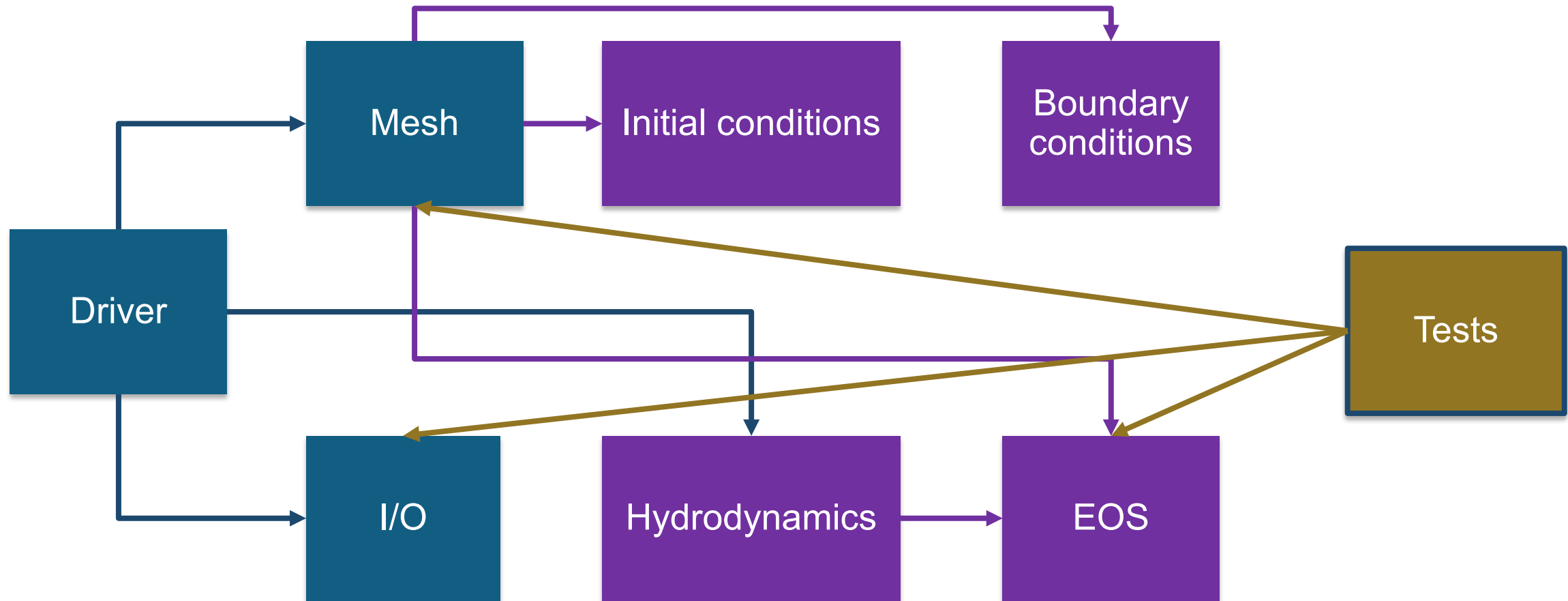
☐ Components evolving with research – physics solvers

- ☐ Initial and boundary conditions
- ☐ Hydrodynamics
- ☐ EOS

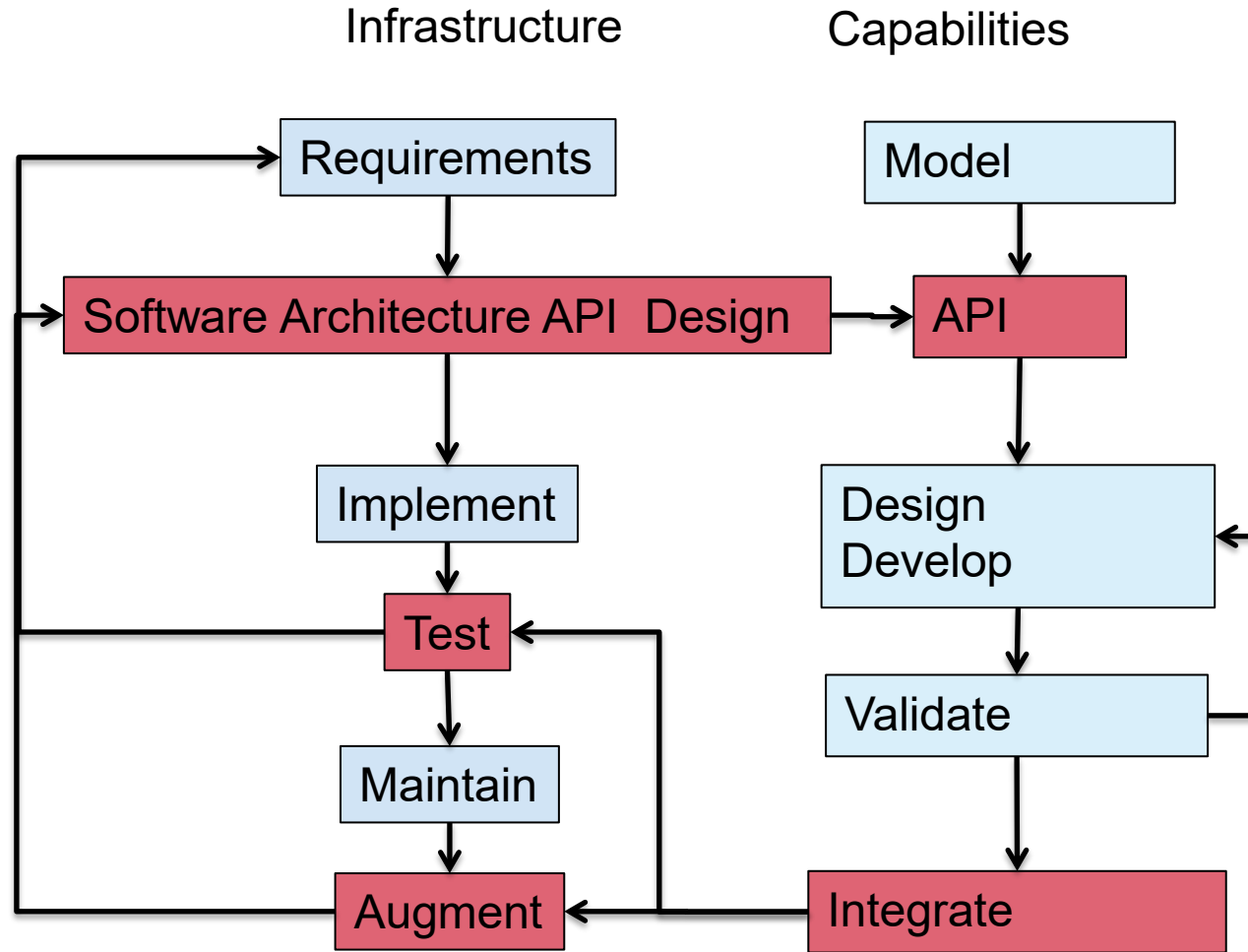
Deeper Dive into some Components

- Driver
 - Iterate over blocks
 - Implement connectivity
- Mesh
 - Data containers
 - Halo cell fill, including application of boundary conditions
 - Reconciliation of quantities at fine-coarse block boundaries
 - Re-mesh when refinement patterns change
- I/O
 - Getting runtime parameters and possibly initial conditions
 - Writing checkpoint and analysis data

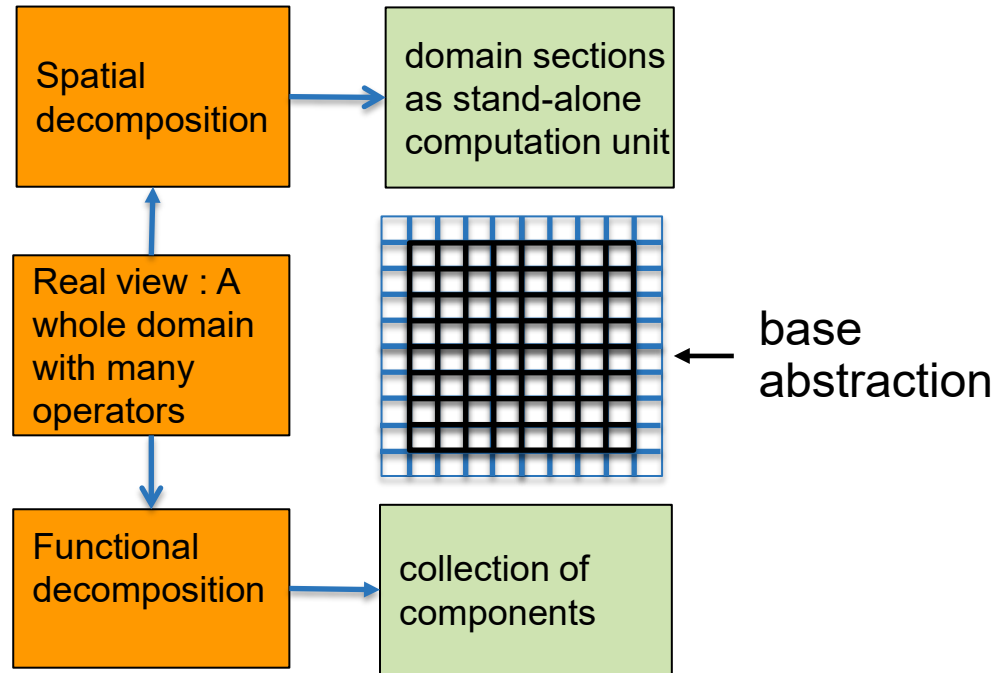
Connectivity



A Design Model for Separation of Concerns



Exploring design space – Abstractions



Constraints

- Only infrastructure components have global view
- All physics solvers have block view only

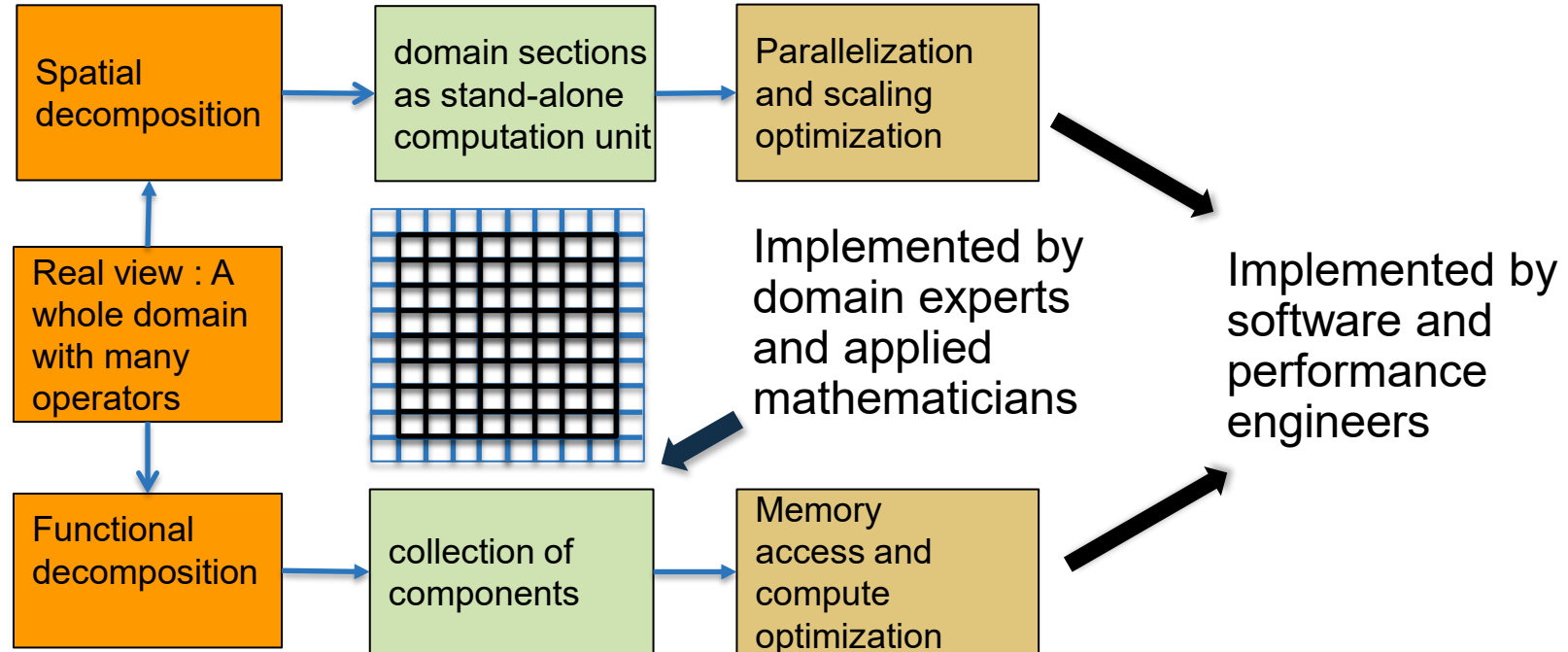
Other Design Considerations

- Data scoping
- Interfaces in the API

Minimal Mesh API

- Initialize_mesh
- Halo_fill
- Access_to_data_containers
- Reconcile_fluxes
- Regrid

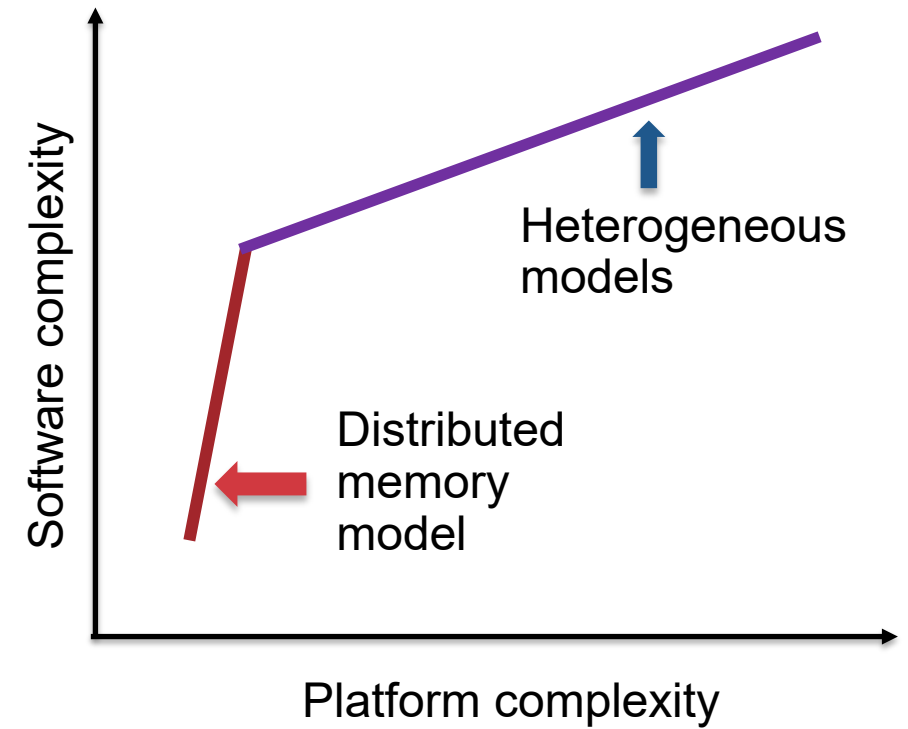
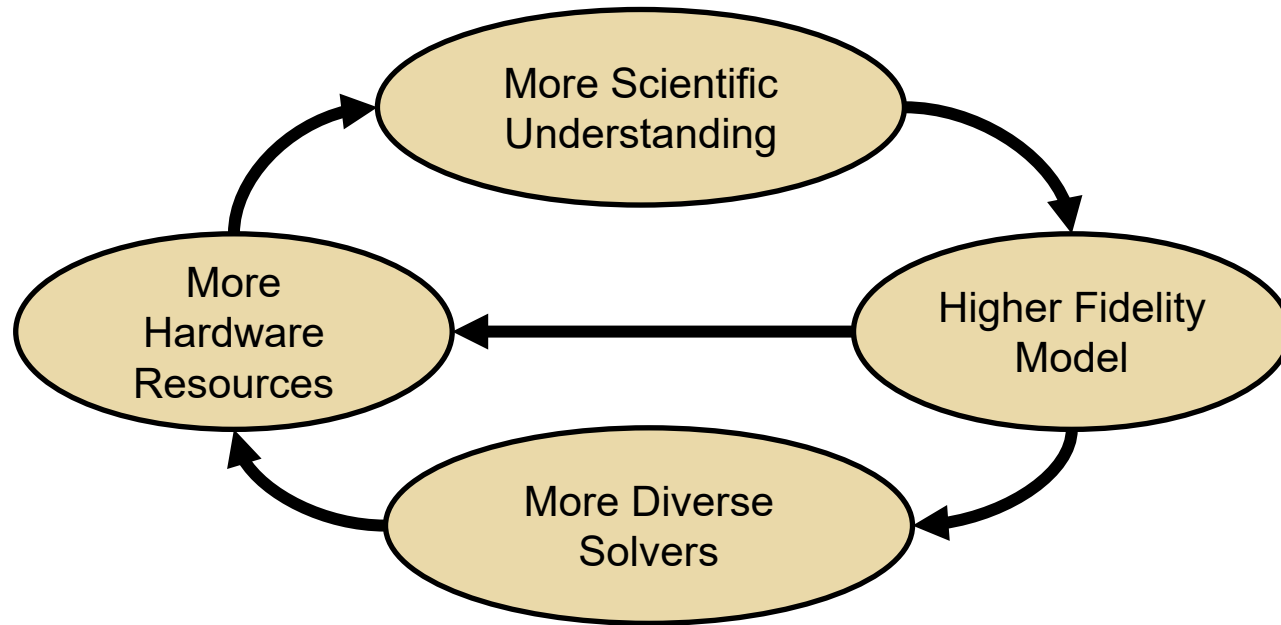
Separation of Concerns Applied



Takeaways so far

- Differentiate between slow changing and fast changing components of your code
- Understand the requirements of your infrastructure
- Implement separation of concerns
- Design with portability, extensibility, reproducibility and maintainability in mind

New Paradigm Because of Platform Heterogeneity



Mechanisms Needed by the Code

Mechanisms to unify expression of computation

- Minimize maintained variants of source suitable for all computational devices
- Reconcile differences in data structures

Mechanisms to move work and data to computational targets

- Moving between devices
 - Launching work at the destination
 - Hiding latency of movement
- Moving data off node

Mechanisms to map work to computational targets

- Figuring out the map
 - Expression of dependencies
 - Cost models
- Expressing the map

So, what do we need?

- Abstractions layers
- Code transformation tools
- Data movement orchestrators

Mechanisms Needed by the Code: Example of Flash-X

Mechanisms to unify expression of computation

Macros with inheritance

Mechanisms to move work and data to computational targets

Domain specific runtime

Mechanisms to map work to computational targets

DSL for recipes with code generator

Composability in the source
A toolset of each mechanism
Independent tool sets

State of Practice – Abstractions and Runtimes

- Still very focused on GPU
 - Majority of ECP applications park their data on the GPU and just work there
- Abstractions -- data structures and parallelization of loops
- Limitations
 - No way to handle algorithmic variants in a unified way
 - No way to transfer domain knowledge based possible optimizations to the tools
- None of the prevalent languages allow a good way to define data locality
 - Boutique HPC languages like chapel do – but chicken and egg problem with adoption

State of Practice – Abstractions and Runtimes

- Still very focused on GPU
 - Majority of ECP applications park their data on the GPU and just work there
- Abstractions -- data structures and parallelization of loops
- Limitations
 - No way to handle algorithmic variants in a unified way
 - No way to transfer domain knowledge based possible optimizations to the tools
- None of the prevalent languages allow a good way to define data locality
 - Boutique HPC languages like chapel do – but chicken and egg problem with adoption

The holy grail for scientists – write equation and generate code

Very limited success in some domains

Is there another way?

State of Practice – Abstractions and Runtimes

- Still very focused on GPU

- Majority

work there

- Abstraction

s

- Limitation

- No way

- No way

ons to the tools

- None of

fine data locality

- Boutique HPC languages like chapel do – but chicken and egg problem with adoption

**We have been developing one
for Flash-X – started under ECP
and TEAMS, continuing with
RAPIDS and ENAF**

The holy grail for scientists – write equation and generate code

Very limited success in some domains

Is there another way?

ORCHA: Overview

- ORCHA consists of three tools: CG-Kit, Macroprocessor, and Milhoja
- A user expresses the control flow of the simulation in a “recipe”
- CG-Kit reads and parses the given recipe, statically optimizes the control flow graph
- Macroprocessor selects the right definition for the target and translates the static physics code (similar to what Kokkos/AMReX do for C++ codes)
- Milhoja handles data transfers and kernel launching

The magic component that puts everything together – Flash-X-RecipeTools

- These are the application specific parts of ORCHA – need to be developed for each application
- They do code generation for application specific task functions and data packets
- Key design choice to avoid having to change applications intrusively to become compatible with ORCHA

Orthogonal Axes of Challenges and Optimization

- ❑ Have a way of rearranging data locality and moving data and computation
- ❑ Let the human-in-the-loop dictate this

CG-Kit – recipes in python

- templates for different variants
- express where to compute what
- emit code in Fortran/C/C++

- If tools only execute what they are told to, they are simpler
- Code generation is our friend – especially when it is simple forward map
 - And is not entangled with the details of the arithmetic

Milhoja – flatten/decompose data and move it to the target

- combine data into one data packet
- decompose into smaller computational sections if needed

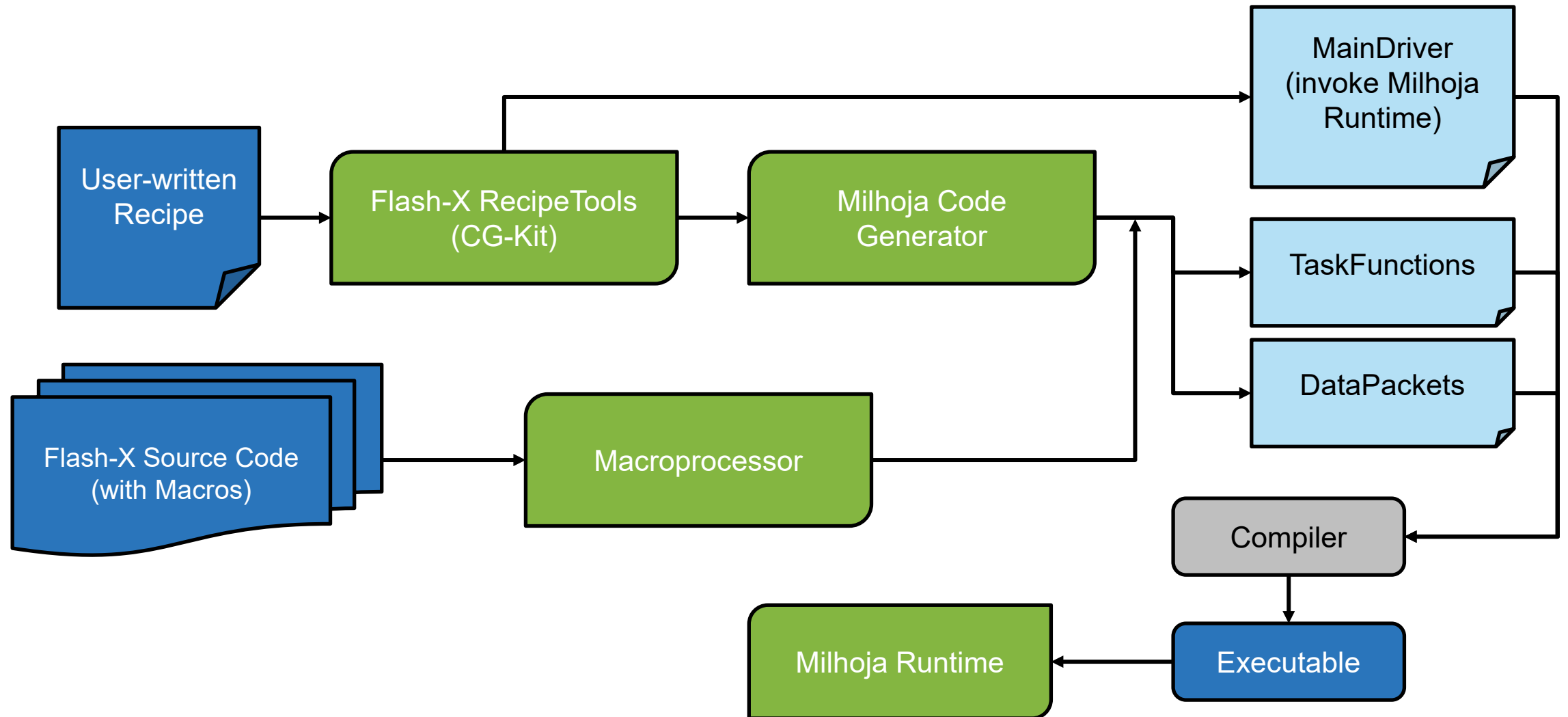
- ❑ If N blocks are sent to the device, we need N copies of all block-wise scratch
- ❑ For all data items we need device pointers
- ❑ Code internally decorated with directives

Code Generators

- Two Classes
 - Data packet generators
 - Parse the interface files
 - Collect all data to be put into a data packet
 - Generate code that will flatten all data into data packets
 - Task function generators
 - Consolidate functions to be invoked
 - Bookended by internode communication
 - Unpack data packets
- Decorate interface definitions with needed metadata

Example -- *this link will work only if you have access to the Flash-X code repository.
Please email flash-x@lists.cels.anl.gov with your github username to get access*

ORCHA: Workflow



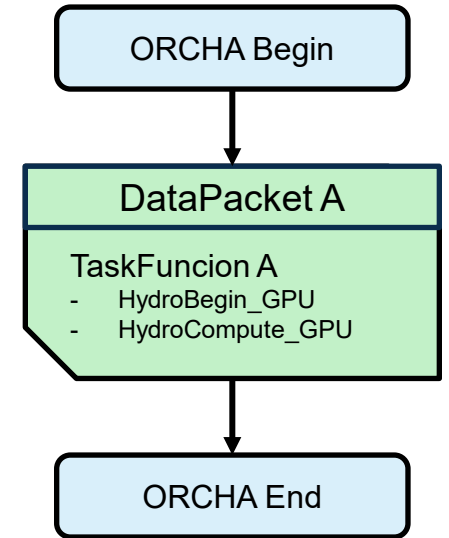
ORCHA: Example 1

- Sedov explosion test
 - Hydrodynamics with an ideal-gas EOS

Recipe (user input):

```
Begin
  addNode(HydroBegin, GPU), dep(Root)
  addNode(HydroCompute, GPU), dep(HydroBegin)
End
```

Generated / transformed codes:



A DataPacket can collect N AMR blocks, where N is a runtime parameter

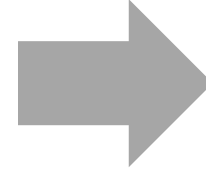
- Hydro exposes 2 interfaces to ORCHA
- Every exposed interface is required to be thread-safe for the whole call-stack, and should be compilable for the target
- In the file where the interfaces are defined, we annotations as comments provide complete inventory of data that is needed for the computation
- Recipetools recursively parse the annotations, cumulate all the data that is part of one taskfunction and generate code for assembling data packets and task functions

ORCHA: Example 2

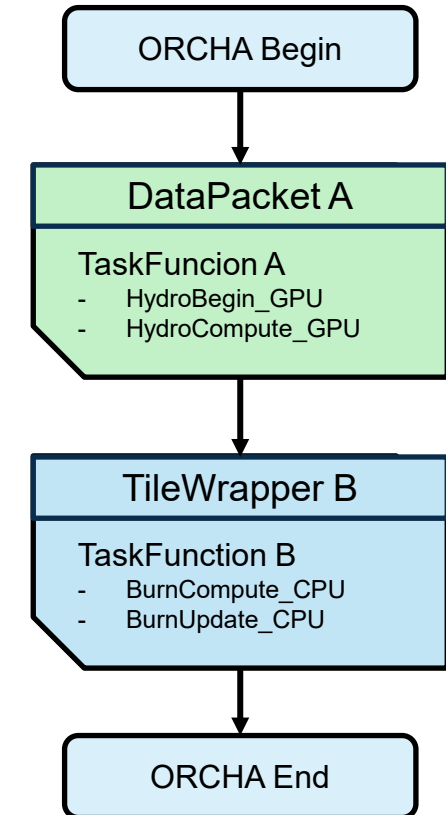
- Cellular detonation
 - Hydrodynamics, nuclear burn, and Helmholtz EOS are needed

Recipe (user input):

```
Begin
  addNode(HydroBegin, GPU), dep(Root)
  addNode(HydroCompute, GPU), dep(HydroBegin)
  addNode(BurnCompute, CPU), dep(HydroCompute)
  addNode(BurnUpdate, CPU), dep(BurnCompute)
End
```



Generated / transformed code:



- Burn exposes 2 interfaces to ORCHA
- Can only run on CPU – third party library dependence
- EOS needs to be compiled for both CPU and GPU, both Hydro and Burn call

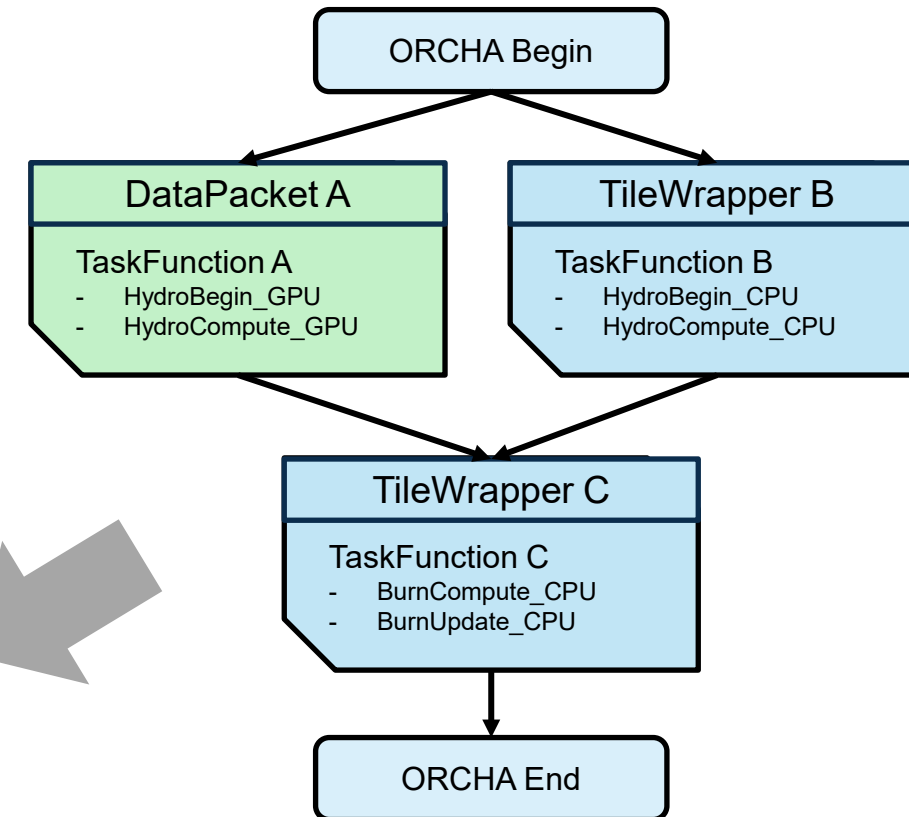
ORCHA: Example 2

- Cellular detonation
 - Hydrodynamics, nuclear burn, and Helmholtz EOS are needed

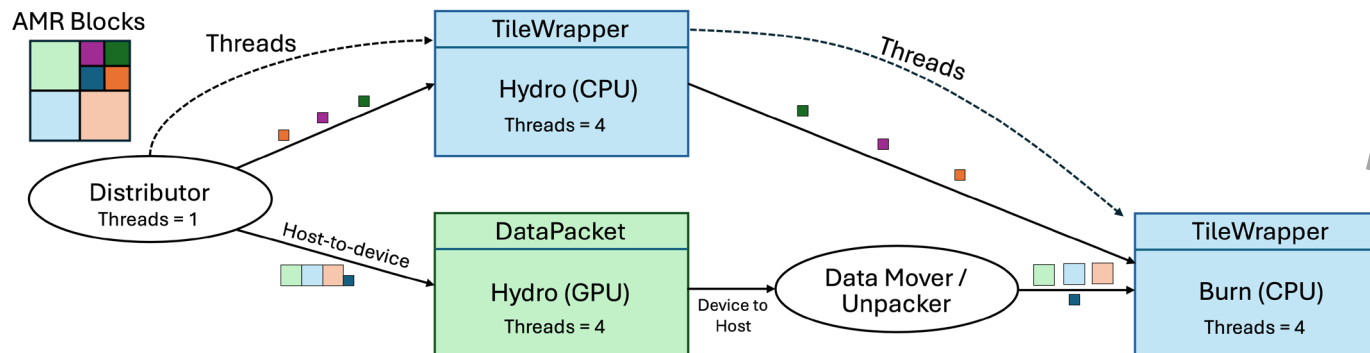
Recipe (user input):

```
Begin
  addNode(HydroBegin, [CPU, GPU]), dep(Root)
  addNode(HydroCompute, [CPU, GPU]), dep(HydroBegin)
  addNode(BurnCompute, CPU), dep(HydroCompute)
  addNode(BurnUpdate, CPU), dep(BurnCompute)
End
```

Generated / transformed code:



Runtime: (GPU + CPU)_{Hydro} → (CPU)_{Burn}



Final takeaways

- Requirements gathering and intentional design are indispensable for sustainable software development
- Many books and online resources available for good design principles
- Research software poses additional constraints on design because of its exploratory nature
 - Scientific research software has further challenges
 - High performance computing research software has even more challenges
 - That are further exacerbated by the ubiquity of accelerators in platforms
- Separation of concerns at various granularities, and abstractions enable sustainable software design

References

- Dubey Anshu, “**Insights from the software design of a multiphysics multicomponent scientific code**” Computing in Science & Engineering, 2021. DOI:[10.1109/MCSE.2021.3069343](https://doi.org/10.1109/MCSE.2021.3069343)
- Dubey, Anshu, et al. "**Flash-X: A multiphysics simulation software instrument.**" *SoftwareX* 19 (2022): 101168. DOI:[10.1016/j.softx.2022.101168](https://doi.org/10.1016/j.softx.2022.101168)
- Rudi, Johann, et al. "**CG-Kit: Code Generation Toolkit for Performant and Maintainable Variants of Source Code Applied to Flash-X Hydrodynamics Simulations.**" *arXiv preprint [arXiv:2401.03378](https://arxiv.org/abs/2401.03378)* (2024).
- O’Neal, Jared, et al. "**Domain-specific runtime to orchestrate computation on heterogeneous platforms.**" *European Conference on Parallel Processing*. Cham: Springer International Publishing, 2021. DOI:[10.1007/978-3-031-06156-1_13](https://doi.org/10.1007/978-3-031-06156-1_13)
- Dubey, Anshu, et al. "**A tool and a methodology to use macros for abstracting variations in code for different computational demands.**" *Future Generation Computer Systems* (2023). DOI:[10.1016/j.future.2023.07.014](https://doi.org/10.1016/j.future.2023.07.014)